Optimizing NCC-Sign for ARMv8

Minwoo Lee[1][0000-0002-2356-3055], Minjoo Sim[1][0000-0001-5242-214X], Siwoo Eum[1][0000-0002-9583-5427], Gyeongju Song[1][0000-0002-4337-1843], and Hwajeong Seo[2][0000-0003-0069-9061]

[1]Department of Information Computer Engineering, Hansung University, Seoul (02876), South Korea,
[2]Department of Convergence Security, Hansung University, Seoul (02876), South Korea,
{minunejip,minjoos9797,shuraatum,thdrudwn98,hwajeong84}@gmail.com

Abstract. This paper presents an ARMv8/NEON–oriented, constant-time implementation of NCC-Sign and a careful head-to-head evaluation against a clean KPQClean baseline on Apple M2. We design lane-local SIMD kernels (e.g., a 4-lane Montgomery multiply–reduce, centered modular reduction, fused stage-0 butterfly, and streamlined radix-2/3 pipelines) that minimize cross-lane shuffles and keep memory access streaming. Using a reproducible harness with PMU-based cycle counting and identical toolchains for baseline and optimized builds, we observe consistent end-to-end gains across all parameter sets: geometric-mean speedups of **1.22**× for key generation, **1.58**× for signing, and **1.50**× for verification, yielding an overall **1.42**× improvement over NCC-Sign-1/3/5. The largest benefits appear in NTT-intensive signing/verification, and absolute savings scale with parameter size.

Keywords: NCC-Sign, ARMv8, NEON, NTT, Montgomery reduction, performance optimization

1 Introduction

Quantum-capable adversaries pose a concrete long-term confidentiality risk: encrypted traffic exfiltrated today may be decrypted in the future once sufficiently large quantum computers exist (the "harvest-now, decrypt-later" threat). This risk is underpinned by Shor's polynomial-time algorithms for factoring and discrete logarithms and by Grover's quadratic search speedup.[1,2] In response, governments and industry have shifted from exploratory research to migration planning toward post-quantum cryptography (PQC), guided by joint readiness advisories and roadmaps.[3]

On the standardization front, NIST has issued its first PQC standards—ML-KEM (FIPS 203), ML-DSA (FIPS 204), and SLH-DSA (FIPS 205)—providing concrete, interoperable targets for near-term deployments.[4,5,6] These documents emphasize not only asymptotic security but also implementation quality, performance, memory footprint, and resistance to side channels in real systems.

In parallel with the NIST effort, Korea organized the KpqC competition to evaluate candidate KEMs and signature schemes for domestic standardization. Among the Round 1 candidates promoted to Round 2 was NCC-Sign, and the final selection announced on January 16, 2025, standardized AIMer and HAETAE for signatures and NTRU+ and SMAUG-T for KEM.[7,8] We nevertheless select NCC-Sign as our study target because its design pursues *non-cyclotomic* rings to reduce exploitable algebraic structure while retaining Bai–Galbraith–style efficiency, with unforgeability proved in the (Q)ROM under RLWE/RSIS/SelfTargetRSIS.[9] From an engineering perspective, the specification highlights uniform (rather than Gaussian) sampling and non-NTT polynomial multiplication (Toom–Cook/Karatsuba) for the non-cyclotomic instantiation, shifting optimization and side-channel trade-offs relative to more conventional cyclotomic designs.[9]

Our focus is an efficient, constant-time software implementation of NCC-Sign on ARMv8 with NEON (Advanced SIMD). ARMv8 is ubiquitous in mobile and edge deployments, and NEON intrinsics expose 128-bit SIMD operations that align well with the structured, data-parallel arithmetic common in lattice-based cryptography. Prior work shows that NEON-tuned ARMv8 implementations can reshuffle performance rankings relative to scalar-C baselines across lattice schemes(e.g., Dilithium/Kyber/Saber, Falcon, and HAETAE)underscoring the broader value of architecture-conscious PQC implementations.[10,11,12]

Accordingly, this paper studies how architecture-agnostic algorithmic kernels can be engineered for ARMv8/NEON to yield end-to-end improvements for a lattice signature workload while preserving constant-time behavior and reproducibility. Section 2 reviews the KpqC competition and standardization context, provides a high-level description of NCC-Sign, and summarizes ARMv8/NEON concepts. Section 3 details our ARMv8-oriented implementation, and Section 4 presents the evaluation. Section 5 concludes.

Contributions. We summarize our main contributions as follows.

- **Optimized Armv8/NEON implementation of NCC-Sign.** We design constant-time, lane-local kernels for the core arithmetic in NCC-Sign, including a 4-lane Montgomery multiply–reduce, centered modular reduction, a fused stage-0 butterfly, and streamlined radix-2/3 pipelines. All kernels avoid cross-lane shuffles and keep memory accesses streaming to fit Apple M2's load/store characteristics (§3).
- **Reproducible measurement methodology.** We use cycle-accurate PMU counts on Apple M2 with overhead subtraction and median-of-n trials after warm-up. The benchmark harness reports end-to-end costs for key generation, signing, and verification, minimizing OS-noise and DVFS effects (§4).
- **End-to-end performance gains.** We obtain consistent improvements across all parameter sets:
  - NCC-Sign-1: **1.21**× (Keygen), **1.53**× (Sign), **1.46**× (Verify).
  - NCC-Sign-3: **1.22**× (Keygen), **1.63**× (Sign), **1.53**× (Verify).
  - NCC-Sign-5: **1.22**× (Keygen), **1.59**× (Sign), **1.51**× (Verify).

Aggregated over the nine benchmarks, this equals a **1.42**× geometric-mean speedup; per-operation geo-means are **1.22**× (Keygen), **1.58**× (Sign), and **1.50**× (Verify) (Table 13, Fig. 2).

– **Open-source release.** We release the full source code, build files, and benchmarking harness to facilitate replication and reuse: https://github.com/minunejip/PQC-ARMv8. The repository includes the Armv8/NEON kernels and scripts to reproduce our tables and figures.

2 Related Work

2.1 KpqC competition and standardization context

The Korean Post-Quantum Cryptography (KpqC) competition was launched to curate a domestically standardized portfolio of KEMs and digital signatures in parallel with international standardization, with an explicit remit that went beyond asymptotic security to include implementation maturity, constant-time discipline, and transition readiness across general-purpose and embedded platforms.

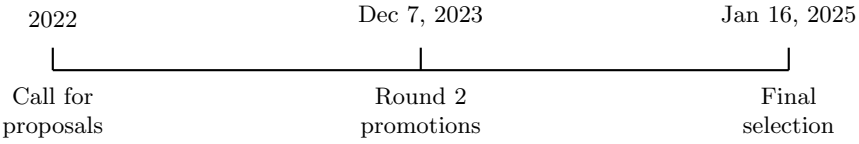

Fig. 1: Schematic timeline of the KpqC competition.

As sketched in Fig. 1, an initial call in 2022 led to a Round 1 cohort comprising nine digital signatures and seven KEMs; on December 7, 2023, eight candidates advanced to Round 2—four signatures (AIMer, HAETAE, MQ-Sign, and NCC-Sign) and four KEMs (NTRU+, PALOMA, REDOG, and SMAUG+TiGER (merged)).[7] The final decision on January 16, 2025 standardized *AIMer* and *HAETAE* for signatures and *NTRU+* and *SMAUG-T* for KEM, reflecting both cryptanalytic assurance and implementability considerations.[8]

Category	Algorithm	Family	Hardness (one line)
Signature	AIMer	Non-lattice (ZK/MITH)	OWF preimage resistance [13]
Signature	HAETAE	Lattice (module)	MLWE/MLSIS [14]
KEM	NTRU+	Lattice (NTRU lineage)	NTRU problem [15]
KEM	SMAUG–T	Lattice (MLWE/MLWR)	MLWE/MLWR [16]

Table 1: KpqC final selections by family and hardness.

To situate these choices within deployment practice, complementary studies report reproducible benchmarking and constant-time code hygiene for Round 1 candidates,[17] and integration of KpqC algorithms into production-grade TLS/X.509 stacks to quantify end-to-end overheads and deployment trade-offs.[18] Within this landscape, NCC-Sign is noted as a Round 2 signature candidate.

2.2 NCC-Sign: A Ring-LWE-Based Digital Signature Scheme

NCC - Sign is a lattice-based digital signature built on the Ring-Learning with Errors (Ring–LWE) assumption. It is intended as a lightweight yet structurally cautious alternative to mainstream lattice signatures such as Dilithium (ML–DSA). Unlike designs fixed to power-of-two cyclotomic rings, NCC - Sign uses a more flexible polynomial ring to reduce exploitable symmetries and to allow finer parameter tuning.[9] The algebraic setting is $R = \mathbb{Z}_q[X]/\phi(X)$ without restricting $\phi(X)$ to be cyclotomic. The specification provides two instantiations: (i) a *non-cyclotomic* option $\phi(X) = X^p - X - 1$ with prime p, which suppresses cyclotomic/subfield structure; and (ii) an *NTT-friendly trinomial* $\phi(X) = X^n - X^{n/2} + 1$ with $n = 2^a 3^b$ for fast transforms.[9]
Parameters. Table 2 lists the conservative non-cyclotomic sets ($\eta = 2$). "Exp. reps." denotes the expected number of signing attempts due to rejection sampling (lower is better).

Level	p	q	d	τ	γ_1	γ_2	η	β	ω	Exp. reps.
I_c	1201	17,279,291	12	32	2^{19}	$(q-1)/70$	2	128	80	2.50
III_c	1607	17,305,741	13	32	2^{19}	$(q-1)/60$	2	128	80	3.02
V_c	2039	17,287,423	13	32	2^{19}	$(q-1)/58$	2	128	80	3.95

Table 2: NCC - Sign non - cyclotomic parameter sets (conservative).

Protocol. NCC - Sign follows the Bai–Galbraith "Fiat–Shamir with aborts" pattern with rounding and hint reconciliation.[19,20] Key generation publishes a compressed $t_1 = \mathrm{HighBits}_q(t, 2^d)$ of $t = as_1 + s_2$ and retains $t_0 = t - t_1 \cdot 2^d$. Signing samples bounded y, computes w and $w_1 = \mathrm{HighBits}_q(w, 2\gamma_2)$, derives a τ-sparse challenge $c \leftarrow \mathrm{SampleInBall}(H(\mu \| w_1))$, sets $z = y + c\,s_1$ and a hint h, and rejects unless $\|z\|_\infty < \gamma_1 - \beta$ and low-bits checks pass. Verification reconstructs $w_1' = \mathrm{UseHint}_q(h,\, az - c\,t_1 \cdot 2^d,\, 2\gamma_2)$ and accepts if $c = H(\mu \| w_1')$ and $\mathrm{wt}(h) \le \omega$. Secrets s_1, s_2 are sampled *uniformly* from bounded sets ($\eta \in \{1, 2\}$), avoiding Gaussian samplers and easing constant-time implementations.
For NTT-friendly settings, Table 3 summarizes the trinomial parameter sets used in practice.

Level	n	q	d	τ	γ_1	γ_2	η	β	ω	Exp. reps.
1	1152	8,401,537	12	25	2^{18}	131,274	1	50	80	1.93
3	1536	8,397,313	12	29	2^{18}	131,208	1	58	80	2.76
$5'$	2048	8,380,417	11	32	2^{18}	130,944	1	64	80	4.49
5	2304	8,404,993	13	32	2^{19}	262,656	1	64	80	2.32

Table 3: NCC - Sign trinomial parameter sets.

Security. Unforgeability in the (Q)ROM is argued under RLWE/RSIS/Self-TargetRSIS using standard Fiat–Shamir-with-aborts techniques; rounding/hint mechanisms follow the established analysis of Lyubashevsky and Bai–Galbraith.[19,20]

Artifact sizes. Table 4 compares public-key, secret-key, and signature sizes across representative PQC schemes; NCC rows correspond to the conservative sets above.[9] Numbers for Dilithium follow FIPS 204 (ML–DSA), SPHINCS+ follows the CCS 2019 paper, and Falcon follows its v1.2 specification.[5,21,22] Rows for HQC and Classic McEliece are KEMs and are included only for key-size context.

Scheme	**Public key**	**Secret key**	**Signature**
Dilithium-II	1,312	2,528	2,420
Dilithium-III	1,952	4,000	3,293
Dilithium-V	2,592	4,864	4,595
NCC - Sign-I_c	1,984	2,800	3,186
NCC - Sign-III_c	2,443	3,914	4,251
NCC - Sign-V_c	3,091	4,940	5,385
Falcon-512	897	1,281	666
Falcon-1024	1,793	2,305	1,280
SPHINCS+-128s	32	64	7,856
SPHINCS+-128f	32	64	17,088
SPHINCS+-192s	48	96	16,224
SPHINCS+-192f	48	96	36,584
SPHINCS+-256s	64	128	29,792
SPHINCS+-256f	64	128	49,856
HQC-128	2,249	2,289	4,497
HQC-192	4,522	4,562	9,042
HQC-256	7,245	7,285	14,485
McEliece-348864	261,120	6,492	96
McEliece-460896	524,160	13,608	156
McEliece-6688128	1,044,992	13,932	194
McEliece-6960119	1,047,319	13,948	208
McEliece-8192128	1,357,824	14,120	208

Table 4: Sizes (bytes) for keys and signatures across selected PQC schemes.

2.3 ARMv8 and NEON SIMD: Concepts and Rationale

This work targets the ARMv8-A execution environment in *AArch64* state. The platform exposes a bank of thirty-two 128-bit SIMD/FP registers $v0$–$v31$ that are shared by Advanced SIMD (*NEON*). Under the AAPCS64 procedure-call standard, $v0$–$v7$ carry vector arguments and return values across public interfaces, $v8$–$v15$ are callee-saved, and all other SIMD registers are caller-saved; these conventions shape inlining boundaries and the way our kernels spill vector state when needed.[23]

NEON provides a fixed-width (128-bit) data-parallel model with integer and floating-point lanes and low-cost permutation primitives (zip/unzip/extract). For lattice-oriented arithmetic (e.g., NTT butterflies, modular reductions, and coefficient reordering), this model enables regular, predictable control flow and memory access, which is desirable for constant-time implementations and cache behavior. In this paper we focus on 32-bit integer lanes and avoid using optional extensions (e.g., SVE/SVE2), to keep portability across mainstream ARMv8-A cores.

We program NEON through the Arm C Language Extensions (*ACLE*) intrinsics (`<arm_neon.h>`), which provide a standardized, compiler-agnostic mapping from C/C++ to A64 vector instructions. Compared to handwritten assembly, intrinsics preserve portability (GCC/Clang/armclang), enable the compiler to schedule around loads/stores, and make constant-time discipline easier to audit; Section 3 details which intrinsics realize each kernel.[24,25]

Empirically, prior work reports substantial speedups from NEON vectorization for lattice schemes on ARMv8-A (e.g., Kyber, Saber, NTRU, Dilithium), confirming that the above programming model is effective for polynomial transforms and modular arithmetic on contemporary cores.[10,26,27]

3 Optimization Methodology

3.1 Montgomery Reduction Kernel (4-Lane NEON)

We implement Montgomery reduction for signed 32-bit lanes with radix $R = 2^{32}$ and an odd modulus $Q < 2^{31}$. Let Q^{-1} satisfy $Q \cdot Q^{-1} \equiv -1 \pmod{2^{32}}$. For a broadcast twiddle z and a 4-lane vector b, the scalar reference

$$\mathrm{red}(a) = \big(a - \big((a \bmod 2^{32}) \cdot Q^{-1}\big) \cdot Q\big) \gg 32 \quad \text{with } a = z \cdot b$$

is realized lane-wise using the NEON "doubling-high" primitive $\mathrm{H}(x, y) = \lfloor 2xy/2^{32} \rfloor$ and an arithmetic halving: we form $h_1 = \mathrm{H}(z, b)$, a correction $t = (b \cdot Q^{-1}) \bmod 2^{32}$, $m = z \cdot t$, then $h_2 = \mathrm{H}(m, Q)$, and output $r = (h_1 - h_2)/2$. This mapping avoids cross-lane traffic, remains constant-time, and—given $|z|, |Q| < 2^{31}$—does not trigger saturation in the doubling-high instruction.

Cost and critical path. A scalar (per-lane) Montgomery step typically requires one low 32-bit multiply, one wide multiply+shift, one extra multiply for the correction, and a subtract ($\approx 3 \sim 4$ integer ops per result depending on ISA sequences). Our vector kernel emits *five* NEON instructions for *four* results: `vqdmulhq_s32`, `vmulq_s32`, `vmulq_s32`, `vqdmulhq_s32`, `vhsubq_s32`.

Table 5: NEON intrinsics used in the 4-lane Montgomery kernel (int32 lanes).

Intrinsic	Operands	Per-lane semantics	Purpose
`vqdmulhq_s32(x,y)`	$(x, y) \to u$	$u = \lfloor 2xy/2^{32} \rfloor$	high$(2xy)$
`vmulq_s32(x,y)`	$(x, y) \to v$	$v = (xy) \bmod 2^{32}$	low 32-bit product
`vhsubq_s32(a,b)`	$(a, b) \to w$	$w = (a - b)$ arith $\gg 1$	halving subtract

This brings the dynamic instruction cost to $5/4 = 1.25$ per result (about 2.4×–3.2× fewer than scalar for a batch of four) with a short dependency chain (*mul→mul→mul→mul→halving-sub*) that overlaps well with butterfly loads/-stores. The algorithmic complexity remains $O(1)$ per residue; the gain is from instruction- and lane-level parallelism.

Algorithm 1 `montgomery_mul4_sqdmulh` (4-lane, NEON; $R = 2^{32}$)

Require: `b`, `vZ`, `vQ`, `vQINV` *(int32x4_t)*
Ensure: `r` $\equiv z \cdot b \cdot R^{-1} \pmod{Q}$

```
high_ab ← vqdmulhq_s32(vZ, b)          ▷ ⌊2zb/2^32⌋ per lane
b_qinv ← vmulq_s32(b, vQINV)           ▷ (b · Q^-1) mod 2^32
a_times ← vmulq_s32(vZ, b_qinv)
high_corr ← vqdmulhq_s32(a_times, vQ)  ▷ ⌊2(a_timesQ)/2^32⌋
r ← vhsubq_s32(high_ab, high_corr)     ▷ arith. halving of (h1 − h2)
return r
```

Engineering notes. (i) `vZ`, `vQ`, and `vQINV` are broadcast on entry (no in-kernel duplication); (ii) each lane is independent, enabling four parallel butterflies; (iii) final normalization (conditional add/sub by Q) can be deferred or fused with butterfly epilogues.

3.2 Centered Modular Reduction (4-Lane `reduce32_vec`)

We normalize signed 32-bit coefficients using a vectorized Barrett-style step tuned for a modulus Q close to 2^{23} (e.g., $Q < 2^{23}$). For a lane value x, we compute

$$t = \Big\lfloor \frac{x + 2^{22}}{2^{23}} \Big\rfloor, \qquad r = x - t \cdot Q.$$

This realizes a centered reduction that returns r in a narrow signed interval around 0 (e.g., $r \in [-Q/2, 3Q/2)$ for inputs that arise inside the NTT pipeline). The operation is branch-free and constant-time; it uses a single right-shift to approximate division by 2^{23} and one multiply by Q for the correction.

Cost and critical path. Per four residues the kernel issues four vector ALU instructions: `vaddq_s32`, `vshrq_n_s32`, `vmulq_s32`, `vsubq_s32`.

Table 6: NEON intrinsics used in `reduce32_vec` (int32 lanes).

Intrinsic	Operands	Per-lane semantics	Purpose
`vaddq_s32(x,c)`	$(x, c) \to u$	$u = x + c$	bias by 2^{22}
`vshrq_n_s32(u,23)`	$u \to t$	$t = u \gg 23$ (logical)	$t \approx \lfloor (x + 2^{22})/2^{23} \rfloor$
`vmulq_s32(t,Q)`	$(t, Q) \to v$	$v = (t \cdot Q) \bmod 2^{32}$	correction $t \cdot Q$
`vsubq_s32(x,v)`	$(x, v) \to r$	$r = x - v$	centered remainder

This is exactly $4/4 = 1.0$ instruction per result, compared to a scalar reduction that typically needs 2–3 integer ops (add/shift/mul/sub) per result. Constants (2^{22} and Q) are broadcast once per call; in hot loops they are hoisted and reused across iterations. The kernel is $O(1)$ per residue; the gain comes from lane-level parallelism and the absence of branches.

Algorithm 2 `reduce32_vec` (4-lane, NEON; centered reduction)

Require: `x` *(int32x4_t)*, constants Q, $C = 2^{22}$
Ensure: `r` $\approx x \bmod Q$ in a centered interval

1: `add` $\leftarrow$ `vdupq_n_s32(`C`)` ▷ broadcast 2^{22}
2: `qv` $\leftarrow$ `vdupq_n_s32(`Q`)` ▷ broadcast Q
3: `t` $\leftarrow$ `vaddq_s32(x, add)` ▷ $x + 2^{22}$
4: `t` $\leftarrow$ `vshrq_n_s32(t, 23)` ▷ $t = \lfloor (x + 2^{22})/2^{23} \rfloor$
5: `tq` $\leftarrow$ `vmulq_s32(t, qv)` ▷ $t \cdot Q$ (low 32 bits)
6: `r` $\leftarrow$ `vsubq_s32(x, tq)` ▷ centered subtraction
7: **return** `r`

Engineering notes. (i) The reduction is applied mid-pipeline (e.g., between radix-2 upper/lower) to cap coefficient growth and keep subsequent multiplications within the non-saturating range of `vqdmulhq_s32`; (ii) hoist `add` and `qv` outside inner loops; (iii) the final normalization to $[0, Q)$ can be deferred to the epilogue or fused with inverse scaling.

3.3 Stage-0 Special Butterfly (4-Lane NEON)

Stage 0 consumes the first twiddle $z =$ `zetas[0]` and applies a fused butterfly on the top split (a, b) with $n_2 = N/2$. Let $t = \mathrm{Mont}(z \cdot b)$ denote the Montgomery product (Sec. 3.1). This variant emits

$$a' = a + t, \qquad b' = a + b - t,$$

which folds an extra addition into the lower output. The mapping is lane-independent and uses one call to the Montgomery kernel per 4 coefficients; no cross-lane operations are required.

Cost and critical path. Per 4-lane pair (producing eight residues, a' and b'): *Loads*: $2 \times$ `vld1q_s32`; *ALU*: Montgomery kernel (5 vector ops) + `vaddq_s32` (for a'), `vaddq_s32` (form $a+b$), `vsubq_s32` (for b') $\Rightarrow$ 8 vector ALU ops; *Stores*: $2 \times$ `vst1q_s32`.

Table 7: NEON intrinsics used in `stage0_neon` (int32 lanes).

Intrinsic	Operands	Per-lane semantics	Purpose
`vld1q_s32(p)`	$p \to x$	load $4 \times$ `int32`	load a, b
`vaddq_s32(x,y)`	$(x, y) \to u$	$u = x + y$	$a{+}t$, $a{+}b$
`vsubq_s32(x,y)`	$(x, y) \to v$	$v = x - y$	$(a{+}b) - t$
`vst1q_s32(p,x)`	$x \to p$	store $4 \times$ `int32`	write a', b'
Also uses `montgomery_mul4_sqdmulh` (Sec. 3.1).			

Arithmetic cost is thus $8/8 = 1.0$ vector ALU per residue, with a short chain that schedules well against the two loads and two stores. Broadcasts of z, Q, and Q^{-1} are hoisted outside the loop.

Algorithm 3 `stage0_neon` (4-lane, NEON; special butterfly)

Require: `Out`, `zetas`, `vQ`, `vQINV`
Ensure: overwrite `Out[0..N/2)` $\leftarrow a'$, `Out[N/2..N)` $\leftarrow b'$

1: $z \leftarrow$ `zetas`[0]; `vZ` $\leftarrow$ `vdupq_n_s32(`z`)`
2: $n_2 \leftarrow N \gg 1$
3: **for** $j \leftarrow 0$ **to** $n_2 - 1$ **step** 4 **do**
4: `a` $\leftarrow$ `vld1q_s32(&Out[j])`
5: `b` $\leftarrow$ `vld1q_s32(&Out[j +` n_2`])`
6: `t` $\leftarrow$ `montgomery_mul4_sqdmulh(b, vZ, vQ, vQINV)` ▷ $t = \mathrm{Mont}(z \cdot b)$
7: `a_plus_t` $\leftarrow$ `vaddq_s32(a, t)` ▷ a'
8: `a_plus_b` $\leftarrow$ `vaddq_s32(a, b)`
9: `b_new` $\leftarrow$ `vsubq_s32(a_plus_b, t)` ▷ b'
10: `vst1q_s32(&Out[j], a_plus_t)`
11: `vst1q_s32(&Out[j +` n_2`], b_new)`

Engineering notes. (i) The broadcast `vZ` is computed once; (ii) loads and stores are contiguous and n_2-strided, enabling coalesced memory access on a/b halves; (iii) Stage 0 keeps values in the range needed to avoid saturation in the subsequent Montgomery steps; a mid-pipeline `reduce32_vec` (Sec. 3.2) after several stages further caps growth.

3.4 Radix-2 Butterfly Pipeline (Upper/Lower, 4-Lane NEON)

We implement the radix-2 stages with two kernels that share the same lane-local structure and differ only in the span of lengths they cover. The `radix2_upper_neon` kernel processes lengths *len* from $N/4$ down to (but not including) `radix2_redlen_ntt`; the `radix2_lower_neon` kernel continues from `radix2_redlen_ntt` down to $3 \cdot$ `radix3_len` (inclusive), after which the mixed radix-3 stage takes over. For each (a, b) pair at a given *len*, we compute

$$a' = a + t, \qquad b' = a - t, \qquad t = \mathrm{Mont}(z \cdot b),$$

Table 8: NEON intrinsics used in radix-2 kernels (int32 lanes).

Intrinsic	Operands	Per-lane semantics	Purpose
`vld1q_s32(p)`	$p \to x$	load 4×int32	load a, b
`vqdmulhq_s32(x,y)`	$(x, y) \to u$	$u = \lfloor 2xy/2^{32} \rfloor$	part of `Mont`
`vmulq_s32(x,y)`	$(x, y) \to v$	$v = (xy) \bmod 2^{32}$	part of `Mont`
`vhsubq_s32(a,b)`	$(a, b) \to w$	$w = (a - b)$ arith $\gg 1$	part of `Mont`
`vaddq_s32(x,y)`	$(x, y) \to u$	$u = x + y$	$a + t$
`vsubq_s32(x,y)`	$(x, y) \to v$	$v = x - y$	$a - t$
`vst1q_s32(p,x)`	$x \to p$	store 4×int32	write a', b'

Algorithm 4 `radix2_upper_neon` (4-lane, NEON; $len : N/4 \to$ `radix2_redlen_ntt`+1)

Require: `Out`, `zetas`, `pk`, `vQ`, `vQINV`

for $len \leftarrow N/4$ **down to** `radix2_redlen_ntt`+1 **by** $len \leftarrow len/2$ **do**
  for $start \leftarrow 0$ **to** $N - 1$ **step** $2 \cdot len$ **do**
    $z \leftarrow$ `zetas[ *pk++ ];` `vZ` $\leftarrow$ `vdupq_n_s32(`z`)`
    $j \leftarrow start$; $j\_end \leftarrow start + len$
    while $j + 4 \le j\_end$ **do**
      `a` $\leftarrow$ `vld1q_s32(&Out[j])`
      `b` $\leftarrow$ `vld1q_s32(&Out[j + len])`
      `t` $\leftarrow$ `montgomery_mul4_sqdmulh(b, vZ, vQ, vQINV)`
      `vst1q_s32(&Out[j + len], vsubq_s32(a, t))` ▷ $b' = a - t$
      `vst1q_s32(&Out[j], vaddq_s32(a, t))` ▷ $a' = a + t$
      $j \leftarrow j + 4$
    while $j < j\_end$ **do** ▷ scalar tail
      $a \leftarrow$ `Out`$[j]$; $b \leftarrow$ `Out`$[j + len]$
      $t \leftarrow$ `montgomery_reduce`$((int64\_t)z \cdot b)$
      `Out`$[j + len] \leftarrow a - t$; `Out`$[j] \leftarrow a + t$
      $j \leftarrow j + 1$

where z is the stage-dependent twiddle and Mont is the 4-lane Montgomery kernel in Sec. 3.1. Each inner loop performs a 4-lane vectorized butterfly, followed by a scalar tail if len is not a multiple of four.

Cost and critical path. Per 4-lane pair (producing eight residues), we perform: two loads, the *Montgomery kernel* (5 vector ALU ops), one `vaddq_s32` and one `vsubq_s32`, and two stores: overall 7 vector ALU ops $\Rightarrow 7/8 = 0.875$ ALU ops per residue.

Twiddle broadcasts are hoisted per `start`-block; `vQ` and `vQINV` are hoisted outside the stage loops. Between the upper and lower halves we insert a mid-pipeline centered reduction (Sec. 3.2) to cap growth and maintain the non-saturating range of `vqdmulhq_s32`. All code paths are branch-free w.r.t. data, ensuring constant-time execution.

Algorithm 5 `radix2_lower_neon` (4-lane, NEON; *len* : `radix2_redlen_ntt` $\to 3\cdot$`radix3_len`)

Require: `Out`, `zetas`, `pk`, `vQ`, `vQINV`

for *len* $\leftarrow$ `radix2_redlen_ntt` **down to** $3 \cdot$ `radix3_len` **by** *len* $\leftarrow$ *len*$/2$ **do**
  for *start* $\leftarrow 0$ **to** $N-1$ **step** $2 \cdot$ *len* **do**
    $z \leftarrow$ `zetas[ *pk++ ]`; `vZ` $\leftarrow$ `vdupq_n_s32(`z`)`
    $j \leftarrow$ *start*; *j_end* $\leftarrow$ *start* + *len*
    while $j + 4 \le j\_end$ **do**
      `a` $\leftarrow$ `vld1q_s32(&Out[j])`
      `b` $\leftarrow$ `vld1q_s32(&Out[j + len])`
      `t` $\leftarrow$ `montgomery_mul4_sqdmulh(b, vZ, vQ, vQINV)`
      `vst1q_s32(&Out[j + len], vsubq_s32(a, t))`
      `vst1q_s32(&Out[j], vaddq_s32(a, t))`
      $j \leftarrow j + 4$
    while $j < j\_end$ **do** ▷ scalar tail
      $a \leftarrow$ `Out`$[j]$; $b \leftarrow$ `Out`$[j + len]$
      $t \leftarrow$ `montgomery_reduce`$((int64\_t)z \cdot b)$
      `Out`$[j + len] \leftarrow a - t$; `Out`$[j] \leftarrow a + t$
      $j \leftarrow j + 1$

Engineering notes. (i) `pk` is advanced once per `start`-block, matching the scalar reference schedule; (ii) the 4-lane loop requires `Out` to be 16-byte aligned for best throughput (alignment faults are still handled correctly by `vld1q_s32`/`vst1q_s32`); (iii) when *len* mod $4 \neq 0$, the scalar tail preserves bit-identical results to the reference; (iv) inserting `reduce32_vec` between upper and lower halves (as done in the driver) reduces the probability of any subsequent doubling-high saturation and tightens value ranges for cache-friendly packing.

3.5 Mixed Radix-3 Forward Kernel (4-Lane NEON; sign1/5)

Table 9: NEON intrinsics used in `radix3_forward_mixed_neon` (int32 lanes).

Intrinsic	Operands	Per-lane semantics	Purpose
`vld1q_s32(p)`	$p \to x$	load 4×int32	load a_0, a_1, a_2
`vaddq_s32(x,y)`	$(x, y) \to u$	$u = x + y$	sums t_{12}, t_{34}; outputs
`vsubq_s32(x,y)`	$(x, y) \to v$	$v = x - y$	$a'_2 = a_0 - (t_{12} + t_{34})$
`vst1q_s32(p,x)`	$x \to p$	store 4×int32	write a'_0, a'_1, a'_2
Also uses `montgomery_mul4_sqdmulh` (Sec. 3.1) four times per triplet.			

For the sign1/5 modes we apply mixed radix-3 steps after the radix-2 "lower" span. Let w, w^2 be the cubic roots of unity pre-scaled in Montgomery form as

`Wmont`, `W2mont`. For a triplet (a_0, a_1, a_2) at a given stride len, with stage twiddles z_1, z_2, we compute

$$t_1 = z_1 \cdot a_1, \quad t_2 = z_2 \cdot a_2, \quad u_1 = w \cdot t_1, \quad u_2 = w^2 \cdot t_2,$$

and output

$$a_0' = a_0 + (t_1 + t_2), \qquad a_1' = a_0 + (u_1 + u_2), \qquad a_2' = a_0 - \big((t_1 + t_2) + (u_1 + u_2)\big),$$

where all products are Montgomery products modulo Q with radix $R = 2^{32}$ (Sec. 3.1). The vector kernel maps these operations lane-wise via the same 4-lane Montgomery primitive and plain add/sub; no cross-lane permutation is needed.

Algorithm 6 `radix3_forward_mixed_neon` (4-lane, NEON; sign1/5)

Require: `Out`, `zetas`, `pk`, `vQ`, `vQINV`

`vW` ← `vdupq_n_s32(Wmont)`; `vW2` ← `vdupq_n_s32(W2mont)`
for $len \leftarrow$ `radix3_len` **down to** 1 **by** $len \leftarrow len/3$ **do**
  for $start \leftarrow 0$ **to** $N - 1$ **step** $3 \cdot len$ **do**
    $z_1 \leftarrow$ `zetas[ *pk++ ]`; $z_2 \leftarrow$ `zetas[ *pk++ ]`
    `vZ1` ← `vdupq_n_s32(`z_1`)`; `vZ2` ← `vdupq_n_s32(`z_2`)`
    $j \leftarrow start$; $j\_end \leftarrow start + len$
    while $j + 4 \le j\_end$ **do**
      `a0` ← `vld1q_s32(&Out[j])`
      `a1` ← `vld1q_s32(&Out[j + len])`
      `a2` ← `vld1q_s32(&Out[j + 2*len])`
      `t1` ← `montgomery_mul4_sqdmulh(a1, vZ1, vQ, vQINV)` ▷ $t_1 = z_1 a_1$
      `t2` ← `montgomery_mul4_sqdmulh(a2, vZ2, vQ, vQINV)` ▷ $t_2 = z_2 a_2$
      `t3` ← `montgomery_mul4_sqdmulh(t1, vW, vQ, vQINV)` ▷ $u_1 = w t_1$
      `t4` ← `montgomery_mul4_sqdmulh(t2, vW2, vQ, vQINV)` ▷ $u_2 = w^2 t_2$
      `t12` ← `vaddq_s32(t1, t2)`; `t34` ← `vaddq_s32(t3, t4)`
      `a2n` ← `vsubq_s32(a0, vaddq_s32(t12, t34))` ▷ a_2'
      `a1n` ← `vaddq_s32(a0, t34)` ▷ a_1'
      `a0n` ← `vaddq_s32(a0, t12)` ▷ a_0'
      `vst1q_s32(&Out[j], a0n)`
      `vst1q_s32(&Out[j + len], a1n)`
      `vst1q_s32(&Out[j + 2*len], a2n)`
      $j \leftarrow j + 4$
    while $j < j\_end$ **do** ▷ scalar tail (bit-identical to reference)
      $a0 \leftarrow$ `Out`$[j]$; $a1 \leftarrow$ `Out`$[j + len]$; $a2 \leftarrow$ `Out`$[j + 2 \cdot len]$
      $t1 \leftarrow$ `montgomery_reduce`$((int64\_t)z_1 \cdot a1)$
      $t2 \leftarrow$ `montgomery_reduce`$((int64\_t)z_2 \cdot a2)$
      $t3 \leftarrow$ `montgomery_reduce`$((int64\_t)$`Wmont`$\cdot t1)$
      $t4 \leftarrow$ `montgomery_reduce`$((int64\_t)$`W2mont`$\cdot t2)$
      $t12 \leftarrow t1 + t2$; $t34 \leftarrow t3 + t4$
      `Out`$[j + 2 \cdot len] \leftarrow a0 - (t12 + t34)$
      `Out`$[j + len] \leftarrow a0 + t34$; `Out`$[j] \leftarrow a0 + t12$
      $j \leftarrow j + 1$

Cost and critical path. Per 4-lane triplet (a_0, a_1, a_2) (producing 12 residues): *Loads*: $3 \times$ `vld1q_s32`; *ALU*: four calls to the Montgomery kernel (each 5 vector ALU ops) $\Rightarrow$ 20, plus vector adds/subs for accumulation: `vaddq_s32` (form t_{12}), `vaddq_s32` (form t_{34}), `vaddq_s32`+`vsubq_s32` (for a_2'), `vaddq_s32` (for a_1'), `vaddq_s32` (for a_0') $\Rightarrow$ 5 more, total **25** vector ALU ops; *Stores*: $3 \times$ `vst1q_s32`.

This is $\approx 25/12 \approx 2.08$ vector ALU ops per residue. Twiddle broadcasts `vZ1`, `vZ2`, and constants `vW`, `vW2` are hoisted per-`start` block and reused across iterations. All paths are data-independent (constant-time).

Engineering notes. (i) `vW`, `vW2` are constant across the entire routine and should be hoisted once; (ii) `vZ1`, `vZ2` are broadcast per-`start` block to minimize reloads of the same twiddles; (iii) lengths $len \in \{3, 1\}$ commonly leave a scalar tail; the vector path still handles any full 4-lane chunks; (iv) intermediate ranges fit the non-saturating domain of `vqdmulhq_s32` given $Q < 2^{31}$; a mid-pipeline centered reduction (Sec. 3.2) after radix-2 keeps value growth tight for these steps as well.

3.6 Inverse Path (4-Lane NEON)

The inverse pipeline mirrors the forward path and stays lane-local. For sign1/5 we reuse the same 4-lane Montgomery primitive and compose three pieces in order: (i) an in-place inverse butterfly, (ii) a mixed radix-3 inverse kernel, and (iii) a final fixed-twiddle scaling with (F_1, F_2). For sign3, the driver dispatches to a reference-compatible path.

Cost (summary). `inv_bfly4_store`: about 7 vector ALU ops per 8 residues (≈ 0.875/residue). Radix-3 inverse triplet: about 25 ALU ops per 12 residues (≈ 2.08/residue). Final scaling: about 18 ALU ops per 8 residues (≈ 2.25/residue). All code paths are data-independent (constant-time).

Table 10: NEON intrinsics used in the inverse path (int32 lanes).

Intrinsic	Operands	Per-lane semantics	Role
`vld1q_s32(p)`	$p \to x$	load 4×int32	read lanes
`vaddq_s32(x,y)`	$(x, y) \to u$	$u = x + y$	sum/accumulate
`vsubq_s32(x,y)`	$(x, y) \to v$	$v = x - y$	diff/accumulate
`vdupq_n_s32(c)`	$c \to v$	broadcast scalar	twiddles, F_1, F_2
`vst1q_s32(p,x)`	$x \to p$	store 4×int32	write back
Montgomery primitive: `montgomery_mul4_sqdmulh` (Sec. 3.1).			

Step A: we start with an in-place inverse butterfly that computes `sum` and `diff`, applies one Montgomery product with z^{-1} to the difference, and stores the pair $(a', b') = (\texttt{sum}, t)$.
Step B: next, the mixed radix-3 inverse operates on triplets (a_0, a_1, a_2) using two fixed constants w, w^2 and stage twiddles z_1, z_2; it forms t_1, t_2 and emits (o_0, o_1, o_2) as in the scalar reference.

Algorithm 7 `inv_bfly4_store` (4-lane, NEON)

Require: `a_ptr`, `b_ptr`, `vZinv`, `vQ`, `vQINV`

`a` ← `vld1q_s32(a_ptr)`; `b` ← `vld1q_s32(b_ptr)`
`sum` ← `vaddq_s32(a, b)`
`diff` ← `vsubq_s32(a, b)`
`t` ← `montgomery_mul4_sqdmulh(diff, vZinv, vQ, vQINV)`
`vst1q_s32(a_ptr, sum)` ▷ $a' =$ `sum`
`vst1q_s32(b_ptr, t)` ▷ $b' = t = 0$

Algorithm 8 `radix3_inverse_mixed_neon` (4-lane, NEON; sign1/5)

Require: `out`, `zetas_inv`, `pk`, `vQ`, `vQINV`

`vW` ← `vdupq_n_s32(Wmont)`; `vW2` ← `vdupq_n_s32(W2mont)`
for $len \leftarrow 1$ **to** `radix3_len` **by** $len \leftarrow 3 \cdot len$ **do**
  for $start \leftarrow 0$ **to** $N - 1$ **step** $3 \cdot len$ **do**
    z_1 ← `zetas_inv[ *pk++ ]`; z_2 ← `zetas_inv[ *pk++ ]`
    `vZ1` ← `vdupq_n_s32`(z_1); `vZ2` ← `vdupq_n_s32`(z_2)
    $j \leftarrow start$; $j\_end \leftarrow start + len$
    while $j + 4 \leq j\_end$ **do**
      `a0` ← `vld1q_s32(out + j)`
      `a1` ← `vld1q_s32(out + j + len)`
      `a2` ← `vld1q_s32(out + j + 2*len)`
      `t1a` ← `montgomery_mul4_sqdmulh(a1, vW2, vQ, vQINV)`
      `t1b` ← `montgomery_mul4_sqdmulh(a2, vW, vQ, vQINV)`
      `t1` ← `vaddq_s32(t1a, t1b)`
      `t2` ← `vaddq_s32(a1, a2)`
      `temp`← `vsubq_s32(a0, vaddq_s32(t1, t2))`
      `o2` ← `montgomery_mul4_sqdmulh(temp, vZ2, vQ, vQINV)`
      `o1` ← `montgomery_mul4_sqdmulh(vaddq_s32(a0, t1), vZ1, vQ, vQINV)`
      `o0` ← `vaddq_s32(a0, t2)`
      `vst1q_s32(out + j, o0)`
      `vst1q_s32(out + j + len, o1)`
      `vst1q_s32(out + j + 2*len, o2)`
      $j \leftarrow j + 4$
    while $j < j\_end$ **do** ▷ scalar tail
      $a0 \leftarrow$ `out`$[j]$; $a1 \leftarrow$ `out`$[j + len]$; $a2 \leftarrow$ `out`$[j + 2 \cdot len]$
      $t1$ ← `montgomery_reduce`$((int64\_t)$`W2mont` $\cdot\ a1)$ + `montgomery_reduce`$((int64\_t)$`Wmont` $\cdot\ a2)$
      $t2 \leftarrow a1 + a2$
      `out`$[j + 2 \cdot len] \leftarrow$ `montgomery_reduce`$((int64\_t)z_2 \cdot (a0 - (t1 + t2)))$
      `out`$[j + len] \leftarrow$ `montgomery_reduce`$((int64\_t)z_1 \cdot (a0 + t1))$
      `out`$[j] \leftarrow a0 + t2$
      $j \leftarrow j + 1$

Step C: finally, we apply a fixed $z_0 = \texttt{zetas\_inv}[k]$ (no increment) to the pairwise differences, then scale with (F_1, F_2) via Montgomery products.

Algorithm 9 Final scaling (driver step; sign1/5)

Require: `out`, `zetas_inv`, k, `vQ`, `vQINV`

```
1: n_2 ← N ≫ 1;   vF1 ← vdupq_n_s32(F1);   vF2 ← vdupq_n_s32(F2)
2: vZ0 ← vdupq_n_s32(zetas_inv[k])                          ▷ no increment
3: for j ← 0 to n_2 − 1 step 4 do
4:     a ← vld1q_s32(out + j);   b ← vld1q_s32(out + j + n_2)
5:     sum ← vaddq_s32(a, b);   diff ← vsubq_s32(a, b)
6:     t ← montgomery_mul4_sqdmulh(diff, vZ0, vQ, vQINV)
7:     a_new ← montgomery_mul4_sqdmulh(vsubq_s32(sum, t), vF1, vQ, vQINV)
8:     b_new ← montgomery_mul4_sqdmulh(t, vF2, vQ, vQINV)
9:     vst1q_s32(out + j, a_new)
10:    vst1q_s32(out + j + n_2, b_new)
```

Engineering notes. (i) Broadcasts of Q, Q^{-1}, w, w^2, F_1, F_2, and inverse twiddles are hoisted outside inner loops; (ii) memory access is streaming and in place (two loads/two stores per pair; three per triplet); (iii) a mid-pipeline `reduce32_vec` sweep (Sec. 3.2) can be inserted before Step B or Step C to keep inputs in the non-saturating domain for `vqdmulhq_s32`.

3.7 Normalization and Memory Considerations (4-Lane NEON)

We keep coefficients in a narrow signed range during the pipeline to (i) avoid saturation in the doubling-high path (`vqdmulhq_s32`) and (ii) improve cache behavior by reducing dynamic range before further stages. Instead of normalizing after every butterfly, we apply a *mid-pipeline* centered reduction using `reduce32_vec` (Sec. 3.2), and defer the final normalization to epilogues (e.g., inverse scaling with (F_1, F_2)). This "reduce-late" policy minimizes arithmetic while preserving numerical safety.

Range and safety. Let $t = \mathrm{Mont}(z \cdot b)$. The Montgomery kernel returns a signed representative t congruent to $z \cdot b \cdot R^{-1} \bmod Q$; with $|z|, |Q| < 2^{31}$, no saturation occurs inside `vqdmulhq_s32`. Forward radix-2 butterflies compute $(a', b') = (a \pm t)$. Without reduction, magnitude can grow roughly by a factor ≤ 2 per level, which we cap by inserting a `reduce32_vec` sweep between "upper" and "lower" spans (and symmetrically in the inverse path). This keeps all inputs to subsequent Montgomery steps well within the non-saturating domain.

Cost footprint. The centered reduction pass touches N coefficients once with four vector ALU ops per 4 residues (Sec. 3.2), i.e., $O(N)$ work with 1.0 vector ALU op per residue on average. Amortized over all stages, this is significantly cheaper than normalizing after every butterfly.

Table 11: Normalization points.

Location	Do	Why
Mid (radix-2 upper)	growth → sweep `reduce32_vec`	cap range before lower/mixed
Inverse mid	same as forward → sweep `reduce32_vec`	keep inputs non-saturating
Final epilogue	canonicalize → scale (F_1, F_2); optional reduce	normalize interval

Memory and throughput. We operate *in place* with two contiguous loads and two stores per 4-lane butterfly (stage-0/radix-2), and three loads/stores per radix-3 triplet.

Table 12: NEON primitives relevant to normalization and data movement.

Intrinsic	Operands	Semantics (per lane)	Use
`vdupq_n_s32(c)`	$c \to v$	broadcast scalar c	Q, Q^{-1}, twiddles, 2^{22}
`vld1q_s32(p)`	$p \to x$	load 4×int32	stream a, b, $a_0..a_2$
`vst1q_s32(p,x)`	$x \to p$	store 4×int32	in-place writes
`reduce32_vec(x)`	$x \to r$	centered reduction	mid-pipeline sweep

Twiddles are broadcast once per `start`-block (`vdupq_n_s32`); Q and Q^{-1} are hoisted out of the loops. Accesses over (a, b) halves at stride *len* preserve streaming patterns, and scalar tails handle non-multiples of four without divergence. For best throughput, align `Out` to 16 bytes; `vld1q_s32`/`vst1q_s32` still function correctly if unaligned, with minor penalties.

Algorithm 10 Mid-pipeline centered reduction sweep (driver snippet)

Require: `out` *(int32 array)*, N

```
for i ← 0 to N − 1 step 4 do
    x ← vld1q_s32(out + i)
    x ← reduce32_vec(x)
    vst1q_s32(out + i, x)
```

Engineering notes. (i) Hoist broadcasts of Q, Q^{-1}, and constant twiddles outside inner loops; (ii) keep pointers `restrict` and arrays 16-byte aligned to aid auto-scheduling; (iii) scalar tails (`while (j < j_end)`) are kept branch-regular and match the C reference bit-for-bit; (iv) the single mid-pipeline sweep measurably improves issue efficiency by shortening critical add chains before the next cascade of Montgomery steps.

4 Performance Evaluation

4.1 Benchmarking Environment

We measure the end-to-end cost of the three core signature operations—key generation (`crypto_sign_keypair`), signing (`crypto_sign`), and verification (`crypto_sign_open`/`crypto_sign_verify`)—as these collectively cover the full workload of NCC-Sign and thus capture overall efficiency. All experiments run natively on an Apple M2 with 8 GB RAM under macOS 13.3 and the default power configuration. We compile with `GCC 14.0.3` using `-O3 -Wall -Wextra -march=native -fomit-frame-pointer -Wno-sign-compare -Wno-unused-but-set-variable -Wno-unused-parameter -Wno-unused-result` inside a Visual Studio Code environment.

Cycle counting. For timing we read Apple silicon's PMU core-cycle counter through the private `kperf`/`kpc` interface (`kpc_get_thread_counters()`), following established practice for M1/M2 systems [28,29,30]. Each data point reports the median of multiple trials after a warm-up phase; we subtract a fixed measurement overhead using an empty harness and avoid background load to limit variance.

Baselines. As a reference, we adopt the NCC-Sign implementations from *KPQClean* and its ver. 2 branch [31,32], which follow the PQClean philosophy of clean, dependency-free baselines [33]. KPQClean removes external dependencies (e.g., OpenSSL) and unifies common primitives, enabling fair, apples-to-apples benchmarking across CPUs, including Apple silicon. We build both the baseline and our optimized variant with identical toolchains and flags; the only change is enabling our Armv8/NEON kernels in the optimized build.

Parameter sets. We benchmark the **1**, **3**, and **5** parameter sets of NCC-Sign. In KPQClean, these correspond to the build targets `NCCSign-II`, `NCCSign-III`, and `NCCSign-V`. The mapping and security context follow the official specification [9]. For each set, we will report per-operation cycles and the speedup factor (baseline / optimized).

4.2 Results

Summary. Table 13 reports median cycle counts for NCC-Sign parameter sets 1,35 with the KPQClean baseline and our Armv8/NEON-optimized build. We also show the geometric-mean speedup (baseline/optimized) across parameter sets per operation. Figure 2 visualizes per-operation improvements.

Table 13: Cycle counts (median) with per-parameter speedups on Apple M2 for NCC-Sign-1/3/5. Speedup is shown in parentheses under the *Optimized* value.

Param	Keygen		Sign		Verify	
	Baseline	Optimized	Baseline	Optimized	Baseline	Optimized
NCC-Sign-1	166545	137888 (1.21×)	355311	231496 (1.53×)	216822	148735 (1.46×)
NCC-Sign-3	216841	178021 (1.22×)	450648	276123 (1.63×)	275006	179491 (1.53×)
NCC-Sign-5	332706	271995 (1.22×)	728419	459016 (1.59×)	439860	291103 (1.51×)
Geo-mean speedup	**1.22×**		**1.58×**		**1.50×**	

Discussion. Across all three parameter sets, our Armv8/NEON kernels accelerate the full signing pipeline substantially. For NCC-Sign-1, median cycles drop from 166,545→137,888 (Keygen, 1.21×, −17.2%), 355,311→231,496 (Sign, 1.53×, −34.9%), and 216,822→148,735 (Verify, 1.46×, −31.4%), yielding a cross-operation geometric mean of 1.39×. For NCC-Sign-3, the respective improvements are 216,841→178,021 (1.22×, −17.9%), 450,648→276,123 (1.63×, −38.7%), and 275,006→179,491 (1.53×, −34.7%), for a 1.45× geometric mean. For NCC-Sign-5, we observe 332,706→271,995 (1.22×, −18.3%), 728,419→459,016 (1.59×, −37.0%), and 439,860→291,103 (1.51×, −33.8%), with a 1.43× geometric mean. Aggregating all nine measurements gives an overall geometric-mean speedup of 1.42×.

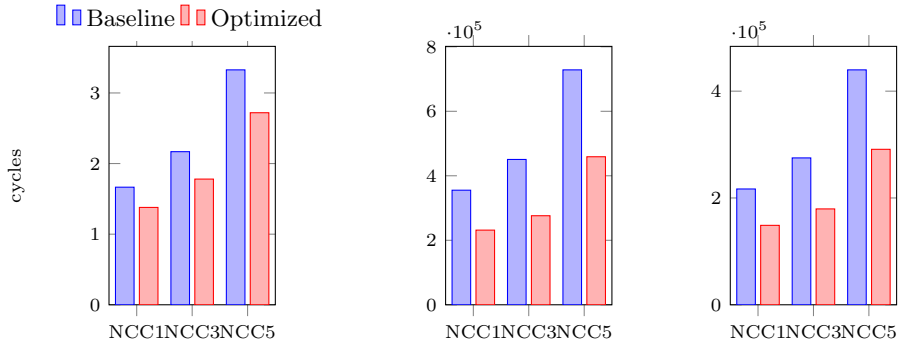

Fig. 2: Per-operation cycle counts: Baseline (left bar) vs. Optimized (right bar) across NCC-Sign-1/3/5.

Where the gains come from. Signing and verification benefit the most (geo-mean 1.58× and 1.50× across parameter sets), which is consistent with these opera-

tions spending the bulk of time in NTT/INTT, pointwise modular multiplications, and reductions. Our vectorized butterflies and four-lane Montgomery products reduce multiply–accumulate latency and instruction count in the hottest loops, while keeping memory access patterns contiguous to leverage Apple M2's load/store bandwidth. Key generation improves more modestly ($\approx$1.22×), as it exercises sampling, packing, and fewer deep NTT passes.

Scaling with parameter size. Absolute cycle savings increase with larger parameter sets (e.g., Sign in NCC-Sign-5 saves 269,403 cycles vs. 123,815 for NCC-Sign-1), reflecting that our SIMD kernels amortize better over longer vectors and deeper stages. The percentage reductions remain stable across sets ($\sim$ 35–39% for Sign; 33–35% for Verify), indicating good scalability of the microarchitectural optimizations.

Methodological notes. All results were collected under identical toolchains and flags for baseline and optimized builds (only NEON kernels differ), using PMU cycle counts with fixed harness-overhead subtraction and median-of-n trials after warm-up. This setup reduces bias from DVFS noise and OS scheduling and aligns with recommended practices for Armv8-A measurement.

5 Conclusion

This paper presented an ARMv8/NEON-oriented implementation of NCC-Sign and a careful evaluation against a clean KPQClean baseline on Apple M2. The core of our approach is a set of constant-time, lane-local kernels—including a 4-lane Montgomery reducer, a centered reduction pass, a fused stage-0 butterfly, radix-2 upper/lower pipelines, and a mixed radix-3 step—that map efficiently to NEON without cross-lane dependencies. Using a reproducible harness and cycle-accurate PMU measurements, we showed consistent end-to-end improvements across all operations and parameter sets: the geometric-mean speedups (baseline/optimized) are **1.22**× for key generation, **1.58**× for signing, and **1.50**× for verification over NCC-Sign-1/3/5, amounting to an overall **1.42**× geo-mean across the nine benchmarks (Table 13, Fig. 2). Beyond relative factors, absolute savings scale with parameter size (e.g., $\approx 2.69 \times 10^5$ cycles saved for signing at NCC-Sign-5), indicating that the kernels amortize well over deeper transforms.

From an engineering perspective, the gains stem from (i) reducing instruction cost per residue in the hottest loops (Montgomery product and add/sub butterflies), (ii) hoisting broadcasts and keeping memory access streaming and contiguous, and (iii) applying a mid-pipeline centered reduction to cap dynamic ranges while preserving constant-time behavior. These techniques are architecture conscious yet scheme agnostic; they readily transfer to NTT-like pipelines and mixed-radix stages in other lattice schemes.

Limitations and future work. Our evaluation targeted Apple M2 and 128-bit NEON; broader profiling across ARMv8-A cores (A55/A76/X-series) and emerging SVE2-capable systems would quantify portability and scalability. Extending

the kernels to wider vectors (SVE2) and exploring micro-architecture–specific scheduling (e.g., load/use spacing, prefetching) may further narrow the gap to peak throughput. Finally, complementary metrics—energy/operation, memory bandwidth pressure, and robustness under system noise—together with leakage-focused testing will give a more holistic picture for deployment.

In summary, we demonstrate that a small set of principled, constant-time SIMD kernels can yield double-digit percentage reductions end-to-end for NCC-Sign on modern ARMv8 systems, without altering the algorithmic interface or the reference control flow. We expect these ideas to be directly useful for other lattice-based PQC schemes and to inform future standardization-oriented implementations on ARM platforms.

References

1. P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
2. L. K. Grover, "Quantum mechanics helps in searching for a needle in a haystack," *Physical Review Letters*, vol. 79, no. 2, pp. 325–328, 1997.
3. CISA, NSA, and NIST, "Quantum-readiness: Migration to post-quantum cryptography," tech. rep., U.S. Cybersecurity and Infrastructure Security Agency, August 2023. (accessed on 16 September 2025).
4. NIST, "Fips 203: Module-lattice-based key-encapsulation mechanism (ml-kem)," tech. rep., National Institute of Standards and Technology, August 2024. (accessed on 16 September 2025).
5. NIST, "Fips 204: Module-lattice-based digital signature standard (ml-dsa)," tech. rep., National Institute of Standards and Technology, August 2024. (accessed on 16 September 2025).
6. NIST, "Fips 205: Stateless hash-based digital signature standard (slh-dsa)," tech. rep., National Institute of Standards and Technology, August 2024. (accessed on 16 September 2025).
7. KpqC Team, "Selected algorithms from the kpqc competition round 1," Dec. 2023. Available online: https://www.kpqc.or.kr/competition.html (accessed on 16 September 2025).
8. KpqC Team, "Selected algorithms from the kpqc competition round 2 (final selection)," Jan. 2025. Available online: https://kpqc.or.kr/competition_02.html (accessed on 16 September 2025).
9. K.-A. Shim, J. Kim, and Y. An, "Ncc-sign: A new lattice-based signature scheme using non-cyclotomic polynomials," tech. rep., National Institute for Mathematical Sciences, 2023. (accessed on 16 September 2025).
10. H. Becker, V. Hwang, M. J. Kannwischer, B.-Y. Yang, and S.-Y. Yang, "Neon ntt: Faster dilithium, kyber, and saber on cortex-a72 and apple m1," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, pp. 221–244, 2022.
11. D. T. Nguyen and K. Gaj, "Fast falcon signature generation and verification using armv8 neon instructions," in *NIST 4th PQC Standardization Conference*, 2022. (accessed on 16 September 2025).
12. M. Sim, M. Lee, and H. Seo, "Haetae on armv8," *Electronics*, vol. 13, no. 19, p. 3863, 2024.

13. S. Kim, J. Cho, M. Cho, J. Ha, J. Kwon, B. Lee, J. Lee, J. Lee, S. Lee, D. Moon, M. Son, and H. Yoon, "The aimer signature scheme: Algorithm specifications," tech. rep., NIST PQC (Digital Signatures Additional Round), 2023. (accessed on 16 September 2025).
14. J. H. Cheon, H. Choe, J. Devevey, T. Güneysu, D. Hong, M. Krausz, G. Land, J. Shin, D. Stehlé, and M. Yi, "Haetae: Algorithm specifications and supporting documentation," tech. rep., KpqC / NIST PQC (Digital Signatures Additional Round), 2023. (accessed on 16 September 2025).
15. J. Kim and J. H. Park, "Ntru+: Compact construction of ntru using simple encoding method," tech. rep., KpqC Submission, 2022. (accessed on 16 September 2025).
16. J. H. Cheon, H. Choe, J. Choi, D. Hong, J. Hong, C.-G. Jung, H. Kang, J. Lee, S. Lim, A. Park, S. Park, H. Seong, and J. Shin, "Smaug-t: the key exchange algorithm based on module-lwe and module-lwr," tech. rep., KpqC Submission, 2024. (accessed on 16 September 2025).
17. Y. Choi, M. Kim, Y. Kim, J. Song, J. Jin, H. Kim, and S. C. Seo, "Kpqbench: Performance and implementation security analysis of kpqc competition round 1 candidates," *IEEE Access*, vol. 12, pp. 18606–18626, 2024.
18. M. Sim, G. Song, S. Eum, M. Lee, S. Yoon, A. Baksi, and H. Seo, "Integrating and benchmarking kpqc in tls/x.509," *Electronics*, vol. 14, no. 18, p. 3717, 2025.
19. V. Lyubashevsky, "Fiat–shamir with aborts: Applications to lattice and factoring-based signatures," in *ASIACRYPT*, vol. 5912 of *LNCS*, pp. 598–616, Springer, 2009.
20. S. Bai and S. D. Galbraith, "Lattice signatures from rejection sampling," in *ASIACRYPT*, vol. 8874 of *LNCS*, pp. 117–137, Springer, 2014.
21. D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, T. Papachristodoulou, J. Rijneveld, and P. Schwabe, "The sphincs+ signature framework," in *ACM CCS*, pp. 2129–2146, 2019.
22. P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-fourier lattice-based compact signatures over ntru. specification v1.2," tech. rep., Falcon Team, 2020. (accessed on 16 September 2025).
23. Arm Limited, "Procedure call standard for the arm 64-bit architecture (aapcs64)," 2025. Available online: https://github.com/ARM-software/abi-aa/releases (accessed on 16 September 2025).
24. Arm Limited, "Arm c language extensions (acle)," 2025. Available online: https://arm-software.github.io/acle/main/acle.html (accessed on 16 September 2025).
25. Arm Limited, "Arm neon intrinsics reference (advanced simd)," 2025. Available online: https://arm-software.github.io/acle/neon_intrinsics/advsimd.html (accessed on 16 September 2025).
26. D. T. Nguyen and K. Gaj, "Fast neon-based multiplication for lattice-based nist post-quantum cryptography finalists," in *Post-Quantum Cryptography (PQCrypto 2021)*, vol. 12710 of *Lecture Notes in Computer Science*, pp. 234–254, Springer, 2021.
27. D. J. Bernstein and P. Schwabe, "Neon crypto," in *Cryptographic Hardware and Embedded Systems – CHES 2012*, vol. 7428 of *Lecture Notes in Computer Science*, pp. 320–339, Springer, 2012. (accessed on 16 September 2025).
28. D. Lemire, "Counting cycles and instructions on arm-based apple systems." https://lemire.me/blog/2023/03/21/

counting-cycles-and-instructions-on-arm-based-apple-systems/, 2023. (accessed on 16 September 2025).
29. ibireme, "A demo showing how to read intel or apple m1 cpu performance counters on macos (kperf/kpc)." https://gist.github.com/ibireme/173517c208c7dc333ba962c1f0d67d12, 2022. (accessed on 16 September 2025).
30. Apple Inc., "Tuning your code's performance for apple silicon." https://developer.apple.com/documentation/apple-silicon/tuning-your-code-s-performance-for-apple-silicon, 2024. (accessed on 16 September 2025).
31. KPQC-CryptoCraft, "Kpqclean: Benchmark on korean post quantum cryptography." https://github.com/kpqc-cryptocraft/KPQClean, 2024. (accessed on 16 September 2025).
32. KPQC-CryptoCraft, "Kpqclean_ver2: Benchmark on kpqc (ver. 2)." https://github.com/kpqc-cryptocraft/KpqClean_ver2, 2024. (accessed on 16 September 2025).
33. M. J. Kannwischer, P. Schwabe, D. Stebila, and T. Wiggers, "Improving software quality in cryptography standardization projects," in *Security Standardization Research (SSR 2022)*, 2022. (accessed on 16 September 2025).